T3000 API & Protocol

Graphics Messages: GET_INITIAL_DATA & LOAD_GRAPHIC_ENTRY · built Nov 13, 2026

Graphics Messages: GET_INITIAL_DATA & LOAD_GRAPHIC_ENTRY

Graphics messages load graphic screen configurations and element data. These messages work together to display HMI screens with interactive elements like text boxes, buttons, and live data displays.

When to Use:

- Loading a graphic screen for display
- Initializing the HMI editor
- Fetching element configurations

Overview

Graphics functionality uses two related messages to load HMI screens:

1. **GET_INITIAL_DATA** - Loads complete graphic screen data
2. **LOAD_GRAPHIC_ENTRY** - Loads individual graphic element

Both messages share similar implementation in BacnetWebView.cpp.

Message: GET_INITIAL_DATA

Request Format

Action: `GET_INITIAL_DATA` (constant defined in message types)

JSON Structure:

```JSON
{
  "action": "GET_INITIAL_DATA",
  "panel_id": 0,
  "graphic_screen_num": 5
}
```

Request Parameters

Parameter	Type	Required	Description
`panel_id`	number	Yes	Target panel ID (0-254)
`graphic_screen_num`	number	Yes	Screen index (1-20)

Response Format

JSON Structure:

JSON


```
{
  "action": "GET_INITIAL_DATA",
  "data": {
    "screen_name": "Main HVAC",
    "background_color": "#FFFFFF",
    "elements": [
      {
        "element_id": 0,
        "type": "TEXT",
        "x": 100,
        "y": 50,
        "width": 200,
        "height": 30,
        "text": "Zone Temperature",
        "font_size": 14,
        "color": "#000000"
      },
      {
        "element_id": 1,
        "type": "LIVE_VALUE",
        "x": 320,
        "y": 50,
        "width": 80,
        "height": 30,
        "point_type": "IN",
        "point_index": 5,
        "format": "%.1f",
        "color": "#0000FF"
      }
    ]
  }
}
```


Implementation

Location: BacnetWebView.cpp line 1227

C++

```cpp
case GET_INITIAL_DATA:
case LOAD_GRAPHIC_ENTRY:
{
    int panel_id;
    int graphic_screen_num;

    // Parse request
    nlohmann::json json_res = nlohmann::json::parse(message);
    panel_id = json_res["panel_id"];
    graphic_screen_num = json_res["graphic_screen_num"];

    // Validate screen number
    if (graphic_screen_num < 1 || graphic_screen_num > MAX_GRPHICS_20)
    {
        return;  // Invalid screen
    }

    // Build response
    nlohmann::json response_json;
    response_json["action"] = (action == GET_INITIAL_DATA) ?
                              "GET_INITIAL_DATA" :
                              "LOAD_GRAPHIC_ENTRY";

    // Get graphic data from global array
    Control_group_point& graphic =
        g_GraphicConfigData[panel_id].at(graphic_screen_num - 1);

    // Serialize graphic data
    nlohmann::json data_json;
    data_json["screen_name"] = graphic.screen_name;
    data_json["background_color"] = graphic.background_color;

    // Serialize elements
    nlohmann::json elements_array = nlohmann::json::array();
    for (int i = 0; i < graphic.element_count; i++)
    {
        Graphic_element& elem = graphic.elements[i];
        nlohmann::json elem_json;

        elem_json["element_id"] = i;
```

```
        elem_json["type"] = elem.element_type;
        elem_json["x"] = elem.x_position;
        elem_json["y"] = elem.y_position;
        elem_json["width"] = elem.width;
        elem_json["height"] = elem.height;

        // Type-specific fields
        if (elem.element_type == ELEM_TEXT)
        {
            elem_json["text"] = elem.text_content;
            elem_json["font_size"] = elem.font_size;
        }
        else if (elem.element_type == ELEM_LIVE_VALUE)
        {
            elem_json["point_type"] = elem.point_type;
            elem_json["point_index"] = elem.point_index;
            elem_json["format"] = elem.value_format;
        }

        elements_array.push_back(elem_json);
    }

    data_json["elements"] = elements_array;
    response_json["data"] = data_json;

    // Send response
    std::string str_json = response_json.dump();
    SendWebViewMessage(str_json.c_str());
}
```

Message: LOAD_GRAPHIC_ENTRY

Purpose: Identical to GET_INITIAL_DATA - loads complete screen

Note: These two messages share the same implementation. The distinction exists for semantic clarity in frontend code.

Use Case Difference:

- `GET_INITIAL_DATA` - Used when first opening HMI editor
- `LOAD_GRAPHIC_ENTRY` - Used when switching between screens

Graphic Screen Structure

Control_group_point

Location: ud_str.h **Max Screens:** 20 (MAX_GRPHICS_20)

```C++
typedef struct {
    char screen_name[32];          // Screen title
    unsigned char element_count;   // Number of elements (0-64)
    unsigned long background_color; // RGB color
    Graphic_element elements[64];  // Element array
} Control_group_point;
```

Graphic_element

Element Types:

Type	Constant	Description
Text Label	ELEM_TEXT	Static text
Live Value	ELEM_LIVE_VALUE	Real-time point value
Button	ELEM_BUTTON	Clickable control
Image	ELEM_IMAGE	Static graphic
Line	ELEM_LINE	Decorative line
Rectangle	ELEM_RECTANGLE	Shape

C++

```
typedef struct {
    unsigned char element_type;    // Element type constant
    unsigned short x_position;     // X coordinate
    unsigned short y_position;     // Y coordinate
    unsigned short width;
    unsigned short height;

    // Type-specific data
    union {
        struct {  // For ELEM_TEXT
            char text_content[64];
            unsigned char font_size;
            unsigned long text_color;
        };
        struct {  // For ELEM_LIVE_VALUE
            unsigned char point_type;   // BAC_IN, BAC_OUT, etc.
            unsigned char point_index;  // Point number
            char value_format[8];       // Printf format
            unsigned long value_color;
        };
        struct {  // For ELEM_BUTTON
            unsigned char action_type;
            unsigned char target_point;
            char button_text[32];
        };
    };
} Graphic_element;
```

Global Storage

Graphics Configuration Array:

C++

```
// Defined in global_variable.h
vector<vector<Control_group_point>> g_GraphicConfigData;

// Access pattern
Control_group_point& screen = g_GraphicConfigData[panel_id][screen_index];
```

Dimensions:

- Outer vector: panel_id (0-254)
- Inner vector: screen_index (0-19, maps to screen 1-20)

Frontend Usage

Loading Screen for Display

TYPESCRIPT

```typescript
const loadGraphicScreen = async (panelId: number, screenNum: number) => {
  const message = {
    action: 'GET_INITIAL_DATA',
    panel_id: panelId,
    graphic_screen_num: screenNum
  };

  // Send via WebSocket
  sendMessage(JSON.stringify(message));

  // Response handled in message listener
};

// Response handler
webSocket.on('message', (data) => {
  const response = JSON.parse(data);

  if (response.action === 'GET_INITIAL_DATA') {
    const screenData = response.data;

    // Render screen
    renderGraphicScreen(screenData);
  }
});

const renderGraphicScreen = (screenData: any) => {
  // Clear canvas
  canvas.clear();

  // Set background
  canvas.setBackgroundColor(screenData.background_color);

  // Render each element
  screenData.elements.forEach((elem: any) => {
    switch (elem.type) {
      case 'TEXT':
```

```
        canvas.addText(elem.x, elem.y, elem.text, {
          fontSize: elem.font_size,
          color: elem.color
        });
        break;

      case 'LIVE_VALUE':
        // Create live value display with point binding
        canvas.addLiveValue(elem.x, elem.y, {
          pointType: elem.point_type,
          pointIndex: elem.point_index,
          format: elem.format,
          color: elem.color
        });
        break;

      case 'BUTTON':
        canvas.addButton(elem.x, elem.y, elem.width, elem.height, {
          text: elem.button_text,
          onClick: () => handleButtonClick(elem.action_type, elem.target_point)
        });
        break;
    }
  });
};
```

Switching Between Screens

TYPESCRIPT

```
const switchScreen = async (newScreenNum: number) => {
  const message = {
    action: 'LOAD_GRAPHIC_ENTRY',
    panel_id: currentPanelId,
    graphic_screen_num: newScreenNum
  };

  sendMessage(JSON.stringify(message));
};
```

Performance Characteristics

Data Size:

- Small screen (10 elements): ~2 KB JSON
- Large screen (64 elements): ~15 KB JSON
- Average load time: 50-150ms

Caching:

- Graphics loaded on demand
- Frontend should cache loaded screens
- Backend loads from global array (already in memory)

Error Handling

Invalid Screen Number

Request:

```JSON
{
  "action": "GET_INITIAL_DATA",
  "panel_id": 0,
  "graphic_screen_num": 25  // Invalid: max is 20
}
```

Result: No response sent (request silently ignored)

Recommendation: Frontend should validate screen numbers before sending.

Empty Screen

Response:

JSON

```
{
  "action": "GET_INITIAL_DATA",
  "data": {
    "screen_name": "",
    "background_color": "#FFFFFF",
    "elements": []  // Empty array
  }
}
```

Related Messages

SAVE_GRAPHIC_DATA

After loading and modifying a screen, use SAVE_GRAPHIC_DATA to persist changes.

Workflow:

TEXT

```
1. GET_INITIAL_DATA → Load screen for editing
2. User modifies elements in HMI editor
3. SAVE_GRAPHIC_DATA → Save modified screen
```

Live Value Updates

Live value elements display real-time data. Use GET_WEBVIEW_LIST or GET_PANEL_DATA to fetch current point values for display.

Example:

TYPESCRIPT

```typescript
// After loading graphic screen with live value elements
const liveElements = screenData.elements.filter(e => e.type === 'LIVE_VALUE');

// Fetch current values
const message = {
  action: 'GET_WEBVIEW_LIST',
  panel_id: panelId,
  entry_type: 'IN',  // For input points
  entry_index_start: 0,
  entry_index_end: 63
};

// Update live values when data arrives
const updateLiveValues = (pointData: any[]) => {
  liveElements.forEach(elem => {
    if (elem.point_type === 'IN') {
      const value = pointData[elem.point_index].value;
      updateElementValue(elem.element_id, value);
    }
  });
};
```

Element Coordinate System

Origin: Top-left corner (0, 0) **Units:** Pixels **Typical Canvas:** 800×600 or 1024×768

Position Calculation:

TYPESCRIPT

```typescript
const centerElement = (canvasWidth: number, elementWidth: number) => {
  return (canvasWidth - elementWidth) / 2;
};

// Example: Center 200px wide element on 800px canvas
const x = centerElement(800, 200);  // = 300
```

See Also

- SAVE_GRAPHIC_DATA - Saving modified screens
- GET_WEBVIEW_LIST - Fetching live point values

- Data Structures - Graphic structure definitions
- Platform Overview - Architecture